EXPO DESIGN AI — LOCAL IMAGE ENGINE

ARCHITECTURE INSPECTION & COMFYUI INTEGRATION REPORT

Document ID: EXP-RPT-IMG-20261013-01 | **Date:** October 13, 2026 | **Hardware:** RTX 5080 (16GB VRAM) | **Tests:** 353 PASSING

1. Executive Summary

This technical report details the codebase inspection and staged engineering roadmap for integrating ComfyUI into Expo Design AI on Windows 11 Pro. The integration operates 100% offline with zero cloud telemetry and zero automated network weight downloads. It manages the 16 GB VRAM envelope on the NVIDIA RTX 5080 alongside existing Ollama workloads via strict single-job serialization.

2. Current Architecture & Inspection Findings

- **Backend & Serving:** FastAPI application on 127.0.0.1:8090. Preserves existing fallback at /api/ai/render while introducing modular /api/image-engine/* endpoints.
- **Configuration:** config.py already defines render_model (FLUX.2-klein-4B), render_provider (local), and render_host (http://127.0.0.1:8188). Resolved via resolve_role("render").
- **Auth & Security:** JWT HS256 tokens with PBKDF2-SHA256 password hashing. Project boundary enforced via auth.require_project() and services.ingestion.contained_upload_path().
- **Durable Job Queue:** SQLite WAL-backed pattern from services/jobs.py with Windows file locking (msvcrt) and crash recovery.
- **Dependencies:** Python 3.12 environment verified with httpx, websockets, pydantic, Pillow, and aiohttp. ZERO new external packages required.

3. Proposed Module Structure

File	Role & Responsibility
services/image_engine/schemas.py	Pydantic v2 schemas for API contracts, parameter ranges, status.
services/image_engine/comfyui_client.py	Async HTTP + WebSocket client for /prompt, /history, /view, and events.
services/image_engine/workflow_registry.py	Workflow loading from workflows/*.json (Prompt API format) & validation.
services/image_engine/job_service.py	SQLite persistent queue, single-job scheduler, OOM error trapping.
services/image_engine/image_storage.py	Project-contained file saving, thumbnails, and metadata sidecars.
services/image_engine/api.py	FastAPI router for status, workflows, job submit, outputs, and cancel.

4. Initial 3 Workflows Specification

- **Workflow A (Architecture Concept Generator):** Text-to-image synthesis from brief, building type, architectural style, dimensions (512x512 to 1024x1024), seed, and steps.
- **Workflow B (Sketch / Reference to Render):** Img2Img concept generation conditioned on uploaded sketch/massing drawing with denoise control (0.3 - 0.8).
- **Workflow C (Render Enhancement):** Photorealistic refinement of 3D exports with upscale/refine node and architectural style grading.

5. Hardware & VRAM Resource Management (RTX 5080 16GB)

- **Strict Serialization:** Single-worker queue (concurrency = 1). Prevents multi-job VRAM thrashing.
- **Safe Checkpoints:** Recommend quantized weights (FLUX.1 [dev] FP8 / GGUF Q4/Q8 or SDXL) fitting in 10-12 GB.
- **OOM Trap:** WebSocket client captures OutOfMemoryError and reports actionable advice without crashing backend.

6. Staged Implementation Plan & Testing Gates

Stage 1: Architecture Inspection & Approved Report (Completed & Verified).
Stage 2: Engine Core, Configuration & ComfyUI Client (feature branch, mock client tests).
Stage 3: Persistent Job Queue & Image Storage (ImageJob/ImageOutput tables, storage tests).
Stage 4: Backend API Integration & Access Control (FastAPI router, require_project guards).

Stage 5: AI Design Studio Frontend (UI panels in app.js/index.html, real-time status banner, gallery).
Stage 6: Real ComfyUI Workflow Integration (JSON API workflows, isolated live GPU test).
Stage 7: Regression Verification & Docs (all 353 baseline tests pass, setup guide).